

TripVault

Travel Memory & Exploration Tracker

Week 2 – Trip Management (CRUD Operations)

A Project Report

NAME : KARTHIK R

Submitted as part of
TripVault Virtual Internship Program

Powered by CodGen

Technology Stack: React.js (Vite) • Node.js • Express.js • MongoDB Atlas • Mongoose • Multer • JWT • bcrypt.js

1. Abstract

This report documents the second milestone of the TripVault Virtual Internship Program, powered by CodGen. Building directly on the authentication system delivered in Week 1, the Week 2 phase introduces Trip Management — a complete Create, Read, Update, and Delete (CRUD) workflow that allows a logged-in traveller to record, review, edit, and remove their trips.

On the backend, a new Trip Mongoose model was introduced with fields for title, destination, start and end dates, a free-text description, a 1–5 star rating, an optional trip photo, and a reference back to the owning user. Five protected REST endpoints were built around this model — `POST /api/trips`, `GET /api/trips`, `GET /api/trips/:id`, `PUT /api/trips/:id`, and `DELETE /api/trips/:id` — each guarded by the JWT authentication middleware from Week 1 and each verifying trip ownership before allowing an update or delete. Image uploads for a trip photo are handled server-side using Multer, accepting multipart/form-data submissions alongside the trip's text fields.

On the frontend, a new Trip Planner page was built in React to list every trip belonging to the signed-in user as a set of trip cards showing the title, destination, travel dates, and star rating. A “Create Trip” action opens a form covering all trip fields including an image picker and a rating selector, and every create, edit, or delete action automatically refreshes the trip list so the UI always reflects the latest state in the database.

This report walks through the objectives and scope of Week 2, the functional and non-functional requirements for trip management, the updated system architecture and database design, the API specification, the folder structure, and the implementation details for both backend and frontend. It documents the algorithms behind the five trip endpoints, presents flowcharts for the create, update, and delete flows, and includes screenshots of the working application, the Swagger API documentation, and the underlying MongoDB collections as evidence of a functioning implementation. The report closes with the results achieved, the advantages of the current build, and a refreshed roadmap of features still to come.

2. Table of Contents

1. Abstract
2. Table of Contents
3. List of Figures
4. List of Tables
5. Recap of Week 1
6. Week 2 Objectives
7. Scope of Week 2
8. Functional Requirements
9. Non-Functional Requirements
10. Software Requirements
11. System Architecture
 - o Layer-by-Layer Explanation
 - o Trip CRUD Workflow
12. Database Design
13. API Endpoints
14. Authentication & Authorization Recap
15. Folder Structure
16. Implementation
 - o Trip Model
 - o Multer Configuration
 - o Trip Controller
 - o Trip Routes
 - o Frontend — Trip Planner Page
 - o Frontend — Trip Form
 - o Testing APIs
17. Component Screenshots
 - o Frontend
 - o API Documentation (Swagger)
 - o Database
18. Algorithms
19. Flowcharts
20. Results
21. Advantages
22. Future Enhancements
23. Conclusion
24. References
25. Appendix

3. List of Figures

Figure 1: System Architecture Diagram (Week 2)	
Figure 2: Trip CRUD Workflow Diagram	
Figure 3: Create Trip Flowchart	
Figure 4: Update Trip Flowchart	
Figure 5: Delete Trip Flowchart	
Figure 6: TripVault Frontend — Dashboard Highlights	
Figure 7: TripVault Frontend — World Exploration & Travel Statistics	
Figure 8: TripVault Frontend — Trip Planner (Trip List)	
Figure 9: TripVault Frontend — Plan New Trip Form	
Figure 10: TripVault Frontend — Travel Journal (Memories)	
Figure 11: TripVault Frontend — New Memory Form	
Figure 12: TripVault Frontend — Login Page	
Figure 13: TripVault Frontend — Profile Page	
Figure 14: Swagger — API Endpoint Overview	
Figure 15: Swagger — GET /api/trips Response	
Figure 16: Swagger — GET /api/trips/{id} Response	
Figure 17: Swagger — POST /api/trips (Create Trip)	
Figure 18: Swagger — DELETE /api/trips/{id} Response	
Figure 19: Swagger — GET /api/memory Response	
Figure 20: Swagger — POST /api/memory (Create Memory)	
Figure 21: Swagger — DELETE /api/memory/{id} Response	
Figure 22: Swagger — POST /api/auth/register	
Figure 23: Swagger — POST /api/auth/login (credentials redacted)	
Figure 24: Swagger — GET /api/auth/me Response	
Figure 25: MongoDB — trips Collection	
Figure 26: MongoDB — memories Collection	

4. List of Tables

Table 1: Trip Collection Schema	
Table 2: Trip API Endpoints	
Table 3: Functional Requirements Summary (Week 2)	
Table 4: Software Additions for Week 2	

5. Recap of Week 1

Week 1 of the TripVault internship established the project's technical foundation: a React.js (Vite) frontend, a Node.js/Express.js backend, a cloud-hosted MongoDB Atlas database, and a complete authentication system in which passwords are hashed with bcrypt.js and sessions are managed statelessly with JWT. Registration, login, and a protected `/api/auth/me` profile route were implemented and verified with Postman.

Week 2 builds directly on top of that foundation. Every trip endpoint introduced in this report sits behind the same authentication middleware built in Week 1, and the React client reuses the same Axios instance and stored JWT to talk to the new `/api/trips` routes.

6. Week 2 Objectives

- To design and implement a Trip Mongoose model capturing title, destination, dates, description, rating, and an optional photo.
- To build a full set of protected CRUD REST endpoints for trips: create, list, retrieve one, update, and delete.
- To ensure every trip is scoped to its owner — a user can only read, edit, or delete their own trips.
- To support trip photo uploads using Multer alongside the existing JWT-protected routes.
- To build a Trip Planner dashboard in React that lists all of a user's trips as cards.
- To build a single reusable trip form that handles both trip creation and trip editing.
- To implement delete-with-confirmation so trips cannot be removed accidentally.
- To automatically refresh the trip list in the UI after every create, update, or delete action.
- To document and test the new endpoints using Swagger / Postman, verifying ownership checks and error handling.

7. Scope of Week 2

The scope of Week 2 is limited to trip management — giving an authenticated user full control over their own list of trips. This includes the Trip schema and its five REST endpoints on the backend, and the Trip Planner page, the Create/Edit Trip form, and delete confirmation on the frontend. It builds on, but does not modify, the authentication system delivered in Week 1.

Also completed during this milestone — and reflected in the screenshots later in this report — is a parallel Memories module (`/api/memory`) that lets a user keep a free-form travel journal alongside their structured trip records, along with a Dashboard page that summarises both trips and memories in one place. Deeper features such as an interactive world map, a full analytics dashboard, and AI-based recommendations remain future work, addressed under Future Enhancements.

8. Functional Requirements

The functional requirements below describe the behaviours the Trip Management module must support.

Requirement	Description
Create Trip	Allow an authenticated user to add a new trip with title, destination, dates, rating, description, and an optional photo.
List Trips	Return every trip belonging to the logged-in user only, never another user's trips.
View Single Trip	Return one trip by its ID, scoped to the requesting user.
Update Trip	Allow the trip's owner to edit any of its fields, including replacing the photo.
Delete Trip	Allow the trip's owner to permanently remove a trip after confirmation.
Ownership Enforcement	Reject update or delete requests for a trip the requester does not own.
Image Upload	Accept a trip photo via multipart/form-data and store its file path on the trip document.
UI Auto-Refresh	Refresh the trip list automatically in the React client after any create, edit, or delete.

Table 3: Functional Requirements Summary (Week 2)

9. Non-Functional Requirements

Performance

Trip list and single-trip queries should return within an acceptable time frame (typically under 300ms under normal load), consistent with the performance target set for authentication in Week 1.

Reliability

Trip documents must persist reliably in MongoDB Atlas, and a failed image upload should not leave a partially-created trip document behind.

Security

Every trip route is protected by JWT verification, and ownership of a trip is re-checked server-side on every update and delete request — never trusted from client input alone.

Scalability

Trip queries are filtered by the authenticated user's ID at the database level, keeping result sets small and query performance stable as the number of users and trips grows.

Maintainability

Trip routes, controller logic, and the Mongoose model follow the same modular MVC-inspired structure introduced in Week 1, keeping the codebase consistent and easy to extend.

Usability

The Trip Planner UI surfaces a clear empty state when a user has no trips yet, and confirms destructive actions (delete) before they are carried out.

10. Software Requirements

Week 2 reuses the full stack established in Week 1 (React.js/Vite, Node.js, Express.js, MongoDB Atlas, JWT, bcrypt.js) and adds the following:

Software / Library	Purpose
Multer	v1.x — Node.js middleware for handling multipart/form-data trip-photo uploads
Mongoose	v7.x/v8.x — schema definition and querying for the new Trip model
Swagger UI	Interactive API documentation and manual testing of the Trip and Memory endpoints
React Router	Client-side routing between Dashboard, Trips, Memories, and Profile pages
Axios	HTTP client reused from Week 1 for all /api/trips requests, including file uploads

Table 4: Software Additions for Week 2

11. System Architecture

Trip Management extends the layered client-server architecture from Week 1. The React frontend still talks to the Express backend over HTTP using Axios, but requests to /api/trips now carry a trip photo as multipart/form-data when one is attached. On the server, every trip request first passes through the existing JWT authentication middleware, and create/update requests are additionally routed through Multer before reaching the trip controller, which persists the trip — including the stored image path — to MongoDB Atlas.

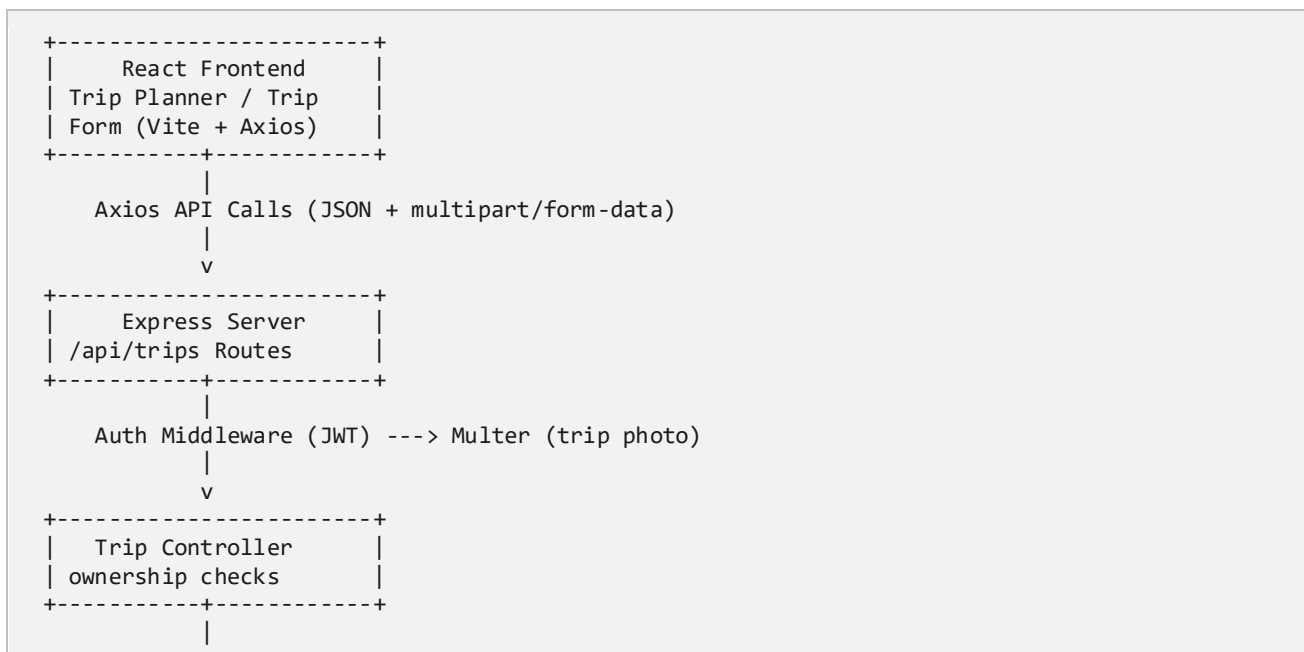




Figure 1: System Architecture Diagram (Week 2)

Layer-by-Layer Explanation

React Frontend

The Trip Planner page and the Create/Edit Trip form render trip cards and collect trip input, including an optional image file, using React state.

Axios API Calls

Axios sends trip requests with the JWT bearer token attached; create and update calls use multipart/form-data so the image file travels alongside the text fields in a single request.

Authentication Middleware

The same JWT-verification middleware from Week 1 protects every /api/trips route, attaching the authenticated user's ID to the request before it reaches Multer or the controller.

Multer

Multer intercepts POST and PUT requests to /api/trips, parses the multipart form body, and makes the uploaded trip photo available to the controller as req.file.

Trip Controller

The controller implements the five CRUD operations, and — critically — re-checks that the trip's user field matches the authenticated user's ID before allowing any update or delete.

MongoDB Atlas

The cloud-hosted database stores each trip as a document in the Trip collection, accessed through a Mongoose model that defines schema validation for every field.

Trip CRUD Workflow

```

[User] --(fills trip form)--> [React Trip Form]
    |
    v
[Axios: multipart/form-data + JWT] --> [Express /api/trips]
    |
    [Auth Middleware: verify JWT]
    |
    [Multer: parse image + fields]
    |
    [Controller: validate + (for update/delete) check ownership]
    |
    [MongoDB Atlas: create / read / update / delete Trip document]
    |
    [Response: JSON trip / trip list / success message]
  
```



```
|
[React: refresh trip list from latest server state]
```

Figure 2: Trip CRUD Workflow Diagram

12. Database Design

Week 2 introduces a second core collection — the Trip collection — sitting alongside the User collection from Week 1.

Field	Type	Description
_id	ObjectId	MongoDB-generated unique identifier for each trip document.
title	String	Required. Short name of the trip, e.g. “Winter in London”.
destination	String	Required. City or country the trip is to.
startDate	Date	Trip start date.
endDate	Date	Trip end date.
description	String	Free-text notes or memories about the trip.
rating	Number	1–5 star rating of how the trip went.
image	String	Stored file path of the uploaded trip photo, set by Multer.
user	ObjectId (ref: 'User')	Links the trip to its owner; used to scope every query.
createdAt / updatedAt	Date	Timestamps automatically maintained by Mongoose.

Table 1: Trip Collection Schema

Field Explanation

- **user:** every query in the controller is filtered by this field, so a user only ever sees or modifies their own trips.
- **rating:** constrained to a 1–5 range in the schema, matching the star-rating control in the frontend form.
- **image:** optional — a trip can be saved without a photo, and Multer only populates this field when a file is attached.
- **startDate / endDate:** stored as native Date types so the frontend can sort and display trips chronologically.

13. API Endpoints

The following RESTful endpoints comprise the Week 2 trip management API surface. All five routes require a valid JWT.

Method	Endpoint	Description
POST	/api/trips	Create a new trip for the logged-in user (supports image upload).
GET	/api/trips	Get all trips belonging to the logged-in user.
GET	/api/trips/:id	Get a single trip by ID (owner only).
PUT	/api/trips/:id	Update an existing trip, including a new photo (owner only).
DELETE	/api/trips/:id	Delete a trip (owner only).

Table 2: Trip API Endpoints

The Memories module, built alongside Trip Management, follows the same pattern under `/api/memory` (POST, GET, PUT, DELETE), also protected by JWT and scoped to the logged-in user.

14. Authentication & Authorization Recap

Every `/api/trips` route is wrapped in the `protect` middleware built in Week 1: it reads the Authorization header, verifies the JWT's signature and expiry, and attaches the decoded user ID to the request as `req.userId`. Trip Management adds a second layer on top of this authentication check — authorization at the resource level. Where Week 1 only asked “is this request signed in?”, the trip controller also asks “does this trip belong to this user?” before permitting an update or delete, returning a 404 (rather than leaking a 403) when a trip exists but is not owned by the requester.

15. Folder Structure

```

Tripvault/
|-- server/
|   |-- models/
|   |   |-- User.js
|   |   |-- Trip.js
|   |   |-- Memory.js
|   |-- routes/
|   |   |-- authRoutes.js
|   |   |-- tripRoutes.js
|   |   |-- memoryRoutes.js
|   |-- middleware/
|   |   |-- authMiddleware.js
|   |   |-- uploadMiddleware.js
|   |-- controllers/
|   |   |-- authController.js
|   |   |-- tripController.js
|   |   |-- memoryController.js
|   |-- config/
|   |   |-- db.js
|   |-- uploads/
|   |-- .env
|   |-- index.js
|-- client/
|   |-- src/
|   |   |-- components/

```

```

|-- pages/
|   |-- Dashboard.jsx
|   |-- Trips.jsx
|   |-- Memories.jsx
|   |-- Profile.jsx
|-- App.jsx

```

Folder Explanation

- server/models/Trip.js — the new Mongoose schema for trips.
- server/routes/tripRoutes.js — maps the five /api/trips HTTP endpoints to controller functions, guarded by the auth middleware.
- server/middleware/uploadMiddleware.js — the new Multer configuration used for trip-photo (and memory-photo) uploads.
- server/controllers/tripController.js — business logic for create, list, retrieve, update, and delete, including ownership checks.
- server/uploads/ — local storage location for uploaded trip and memory photos, served statically.
- client/src/pages/Trips.jsx — the Trip Planner page: trip list, Create Trip button, and the trip form modal.

16. Implementation

Trip Model

The Trip schema mirrors the fields specified for Week 2, with rating constrained to the 1–5 range and a required reference back to the owning user:

```

// models/Trip.js
const mongoose = require('mongoose');

const tripSchema = new mongoose.Schema({
  title: { type: String, required: true },
  destination: { type: String, required: true },
  startDate: { type: Date },
  endDate: { type: Date },
  description: { type: String },
  rating: { type: Number, min: 1, max: 5 },
  image: { type: String },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });

module.exports = mongoose.model('Trip', tripSchema);

```

Multer Configuration

Trip photo uploads are handled with a disk-storage Multer instance that writes to the server's uploads/ folder and generates a collision-resistant filename:

```

// middleware/uploadMiddleware.js

```

```
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => {
    const unique = `image-${Date.now()}${path.extname(file.originalname)}`;
    cb(null, unique);
  },
});

module.exports = multer({ storage });
```

Trip Controller

The controller implements all five CRUD operations. Note that `getTripById`, `updateTrip`, and `deleteTrip` all filter by both `_id` and `user` — a trip that exists but belongs to someone else simply returns 404, the same as a trip that doesn't exist at all:

```
// controllers/tripController.js
const Trip = require('../models/Trip');

exports.createTrip = async (req, res) => {
  const { title, destination, startDate, endDate, description, rating } = req.body;
  const image = req.file ? `/uploads/${req.file.filename}` : undefined;
  const trip = await Trip.create({
    title, destination, startDate, endDate, description, rating, image,
    user: req.userId,
  });
  res.status(201).json(trip);
};

exports.getTrips = async (req, res) => {
  const trips = await Trip.find({ user: req.userId }).sort({ startDate: -1 });
  res.status(200).json(trips);
};

exports.getTripById = async (req, res) => {
  const trip = await Trip.findOne({ _id: req.params.id, user: req.userId });
  if (!trip) return res.status(404).json({ message: 'Trip not found' });
  res.status(200).json(trip);
};

exports.updateTrip = async (req, res) => {
  const trip = await Trip.findOne({ _id: req.params.id, user: req.userId });
  if (!trip) return res.status(404).json({ message: 'Trip not found' });

  const fields = ['title', 'destination', 'startDate', 'endDate', 'description', 'rating'];
  fields.forEach(f => { if (req.body[f] !== undefined) trip[f] = req.body[f]; });
  if (req.file) trip.image = `/uploads/${req.file.filename}`;

  await trip.save();
  res.status(200).json(trip);
};

exports.deleteTrip = async (req, res) => {
```

```
const trip = await Trip.findOneAndDelete({ _id: req.params.id, user: req.userId });
if (!trip) return res.status(404).json({ message: 'Trip not found' });
res.status(200).json({ message: 'Trip removed successfully' });
};
```

Trip Routes

Every route passes through the protect middleware from Week 1; the two routes that accept a photo additionally pass through Multer's `upload.single('image')` first:

```
// routes/tripRoutes.js
const router = require('express').Router();
const protect = require('../middleware/authMiddleware');
const upload = require('../middleware/uploadMiddleware');
const {
  createTrip, getTrips, getTripById, updateTrip, deleteTrip,
} = require('../controllers/tripController');

router.post('/', protect, upload.single('image'), createTrip);
router.get('/', protect, getTrips);
router.get('/:id', protect, getTripById);
router.put('/:id', protect, upload.single('image'), updateTrip);
router.delete('/:id', protect, deleteTrip);

module.exports = router;
```

Frontend — Trip Planner Page

The Trips page fetches the trip list on mount and re-fetches it after every mutating action, so the UI is always driven by the server's current state rather than by optimistic local edits:

```
// pages/Trips.jsx (excerpt)
useEffect(() => { fetchTrips(); }, []);

const fetchTrips = async () => {
  const { data } = await api.get('/api/trips');
  setTrips(data);
};

const handleDelete = async (id) => {
  if (!window.confirm('Delete this trip?')) return;
  await api.delete(`/api/trips/${id}`);
  fetchTrips(); // refresh the list after deleting
};
```

Frontend — Trip Form

A single form component is reused for both creating and editing a trip. It builds a `FormData` payload so the trip photo can travel in the same request as the text fields, and calls `POST` or `PUT` depending on whether a trip is being created or edited:

```
// components/TripForm.jsx (excerpt)
const handleSubmit = async (e) => {
  e.preventDefault();
  const form = new FormData();
  Object.entries(values).forEach(([key, val]) => form.append(key, val));
  if (imageFile) form.append('image', imageFile);

  if (editingTrip) {
    await api.put(`/api/trips/${editingTrip._id}`, form);
  } else {
    await api.post('/api/trips', form);
  }
  onSave(); // parent re-fetches the trip list
};
```

Testing APIs

All five trip endpoints — along with the parallel memory endpoints and the Week 1 auth endpoints — were exercised through the project's Swagger UI at /api-docs, verifying correct status codes, response payloads, and ownership enforcement (a 404 when a trip ID belongs to another user). The full endpoint list and sample requests/responses are captured in the Component Screenshots and Appendix sections below.

17. Component Screenshots

The following screenshots, captured from the working TripVault implementation, illustrate the Week 2 Trip Management feature across the frontend, the API documentation, and the database.

Frontend

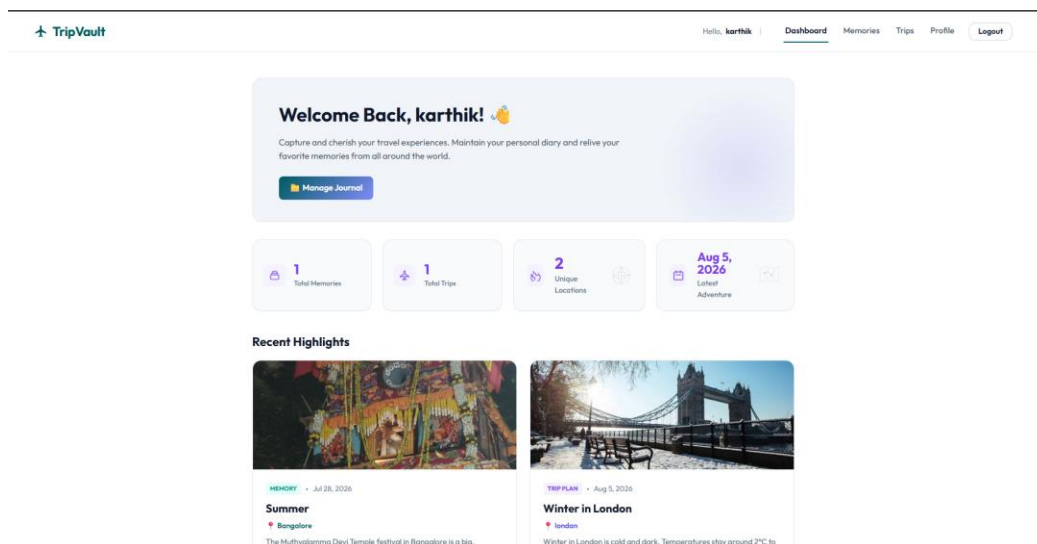


Figure 6: TripVault Frontend — Dashboard Highlights

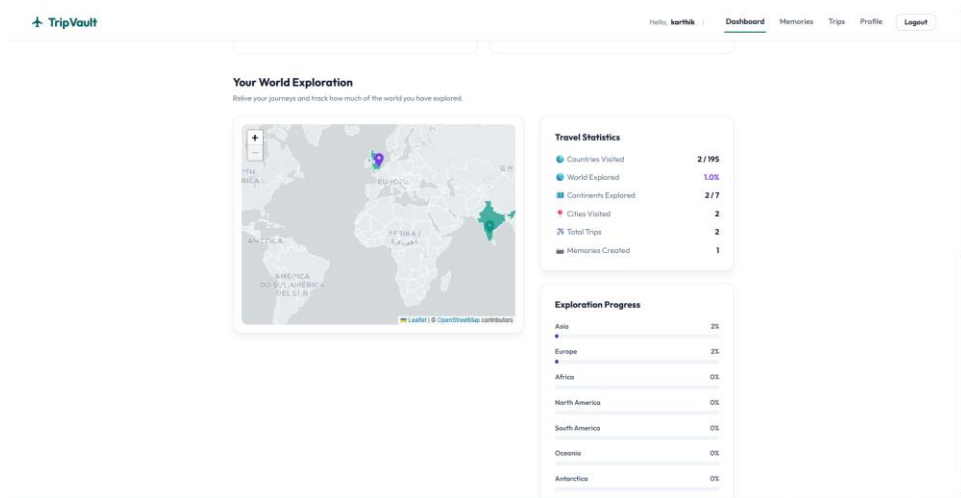


Figure 7: TripVault Frontend — World Exploration & Travel Statistics

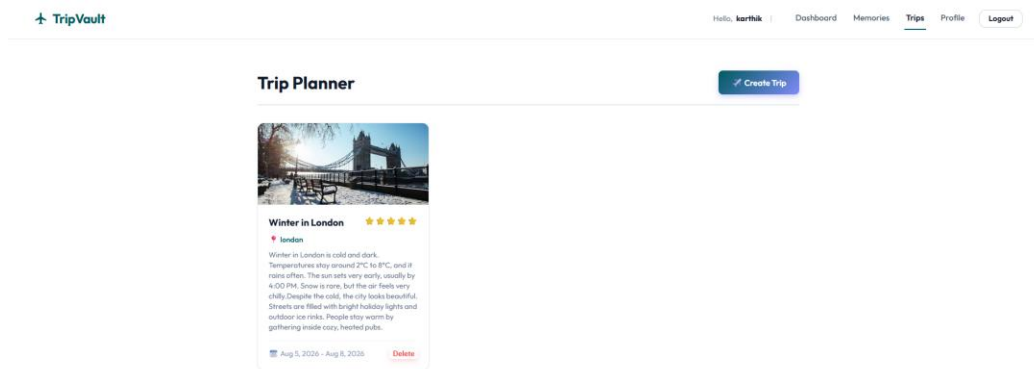


Figure 8: TripVault Frontend — Trip Planner (Trip List)

Plan New Trip ✕

Title

Winter in Switzerland

Destination

Zurich, Switzerland

Start Date **End Date**

13-08-2026 📅 13-08-2026 📅

Rating (1 - 5)

★★★★★ (5/5) ▼

Trip Image

Choose file No file chosen

Description / Notes

Write details about hotels, flights, or highlights...

Add Trip

Figure 9: TripVault Frontend — Plan New Trip Form

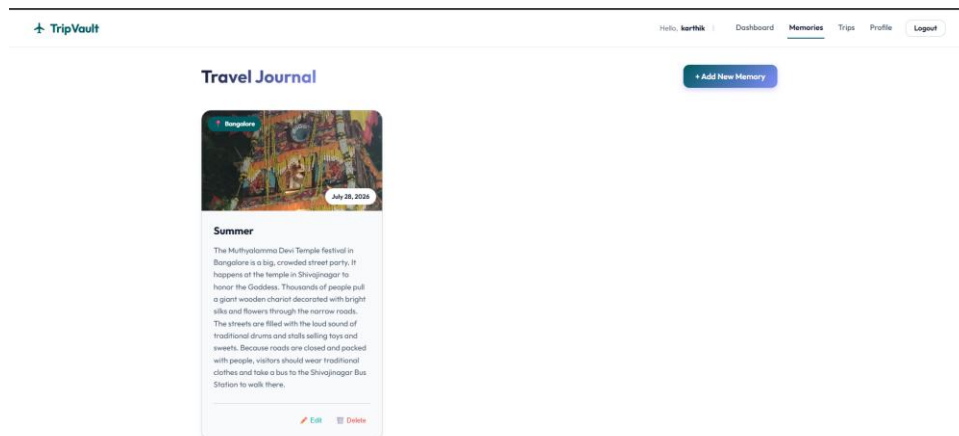


Figure 10: TripVault Frontend — Travel Journal (Memories)

New Memory

Title
Scuba diving in Maldives

Location
Malé, Maldives

Country
Maldives

City
Malé

Start Date
13-08-2026

End Date
13-08-2026

Display Date (Journal Date)
13-08-2026

Description / Diary Entry
Detail your experience, the food, people, and vibes...

Trip Photo
Choose file No file chosen

Figure 11: TripVault Frontend — New Memory Form

TripVault

Welcome Back
Log in to unlock your personal TripVault

Login Register

Email Address
karthik@example.com

Password
password

Login

Don't have an account? Register here

Figure 12: TripVault Frontend — Login Page

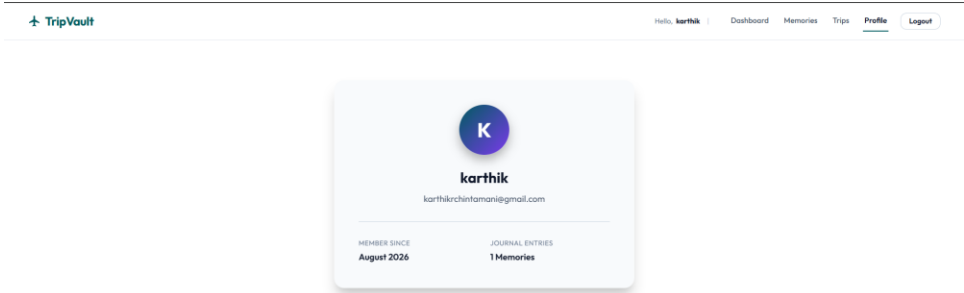


Figure 13: TripVault Frontend — Profile Page

API Documentation (Swagger)

The full set of Trip, Memory, and Authentication endpoints, documented and testable through Swagger UI:

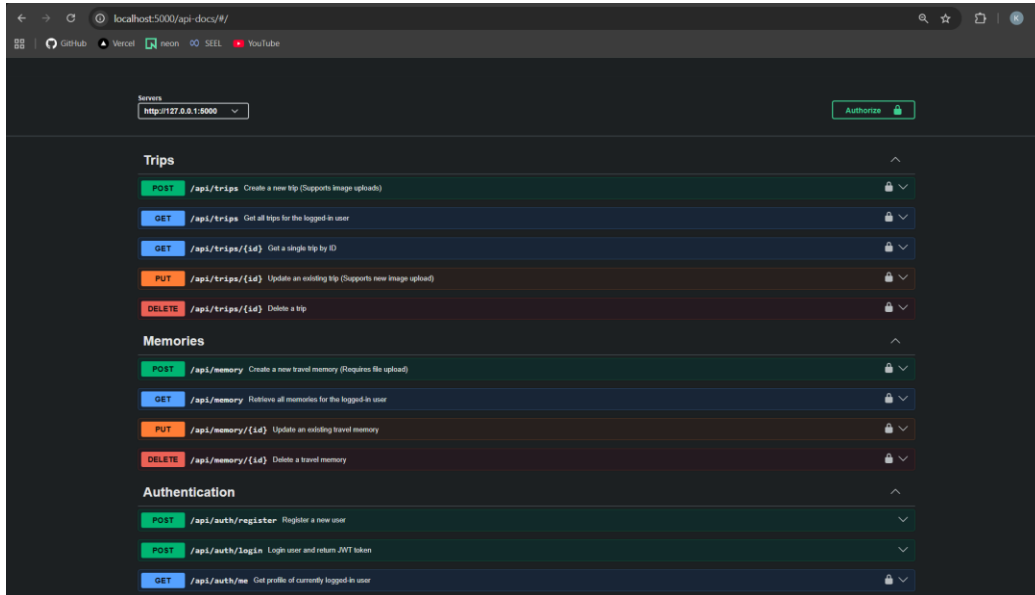


Figure 14: Swagger — API Endpoint Overview

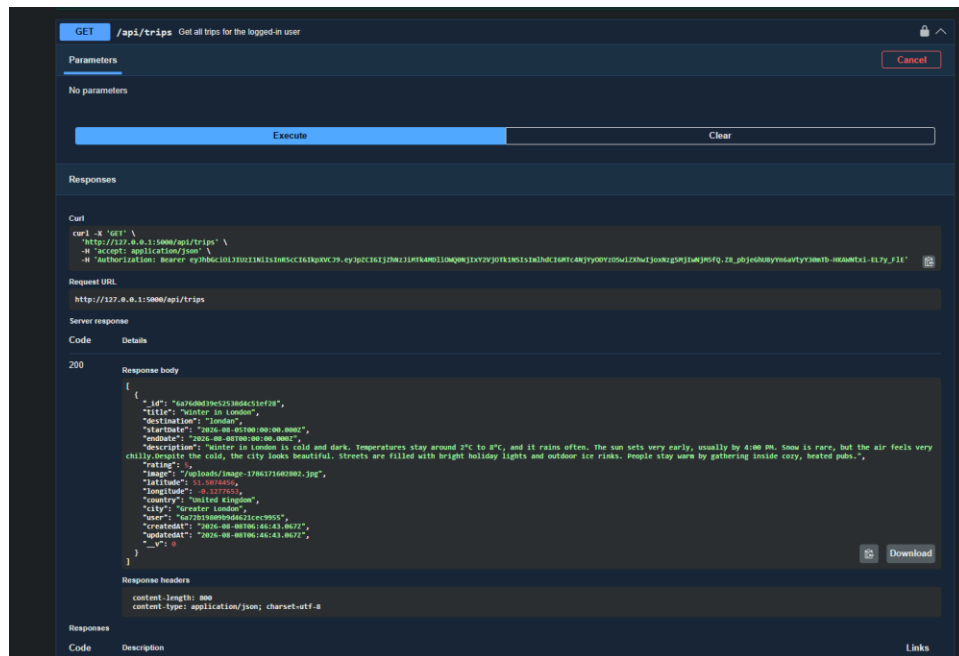


Figure 15: Swagger — GET /api/trips Response

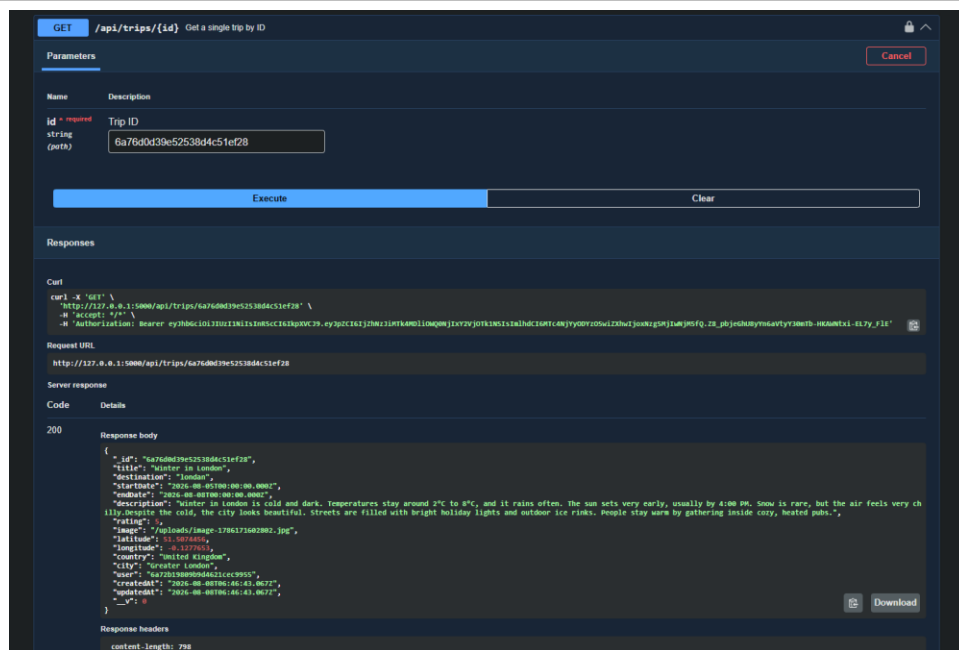


Figure 16: Swagger — GET /api/trips/{id} Response

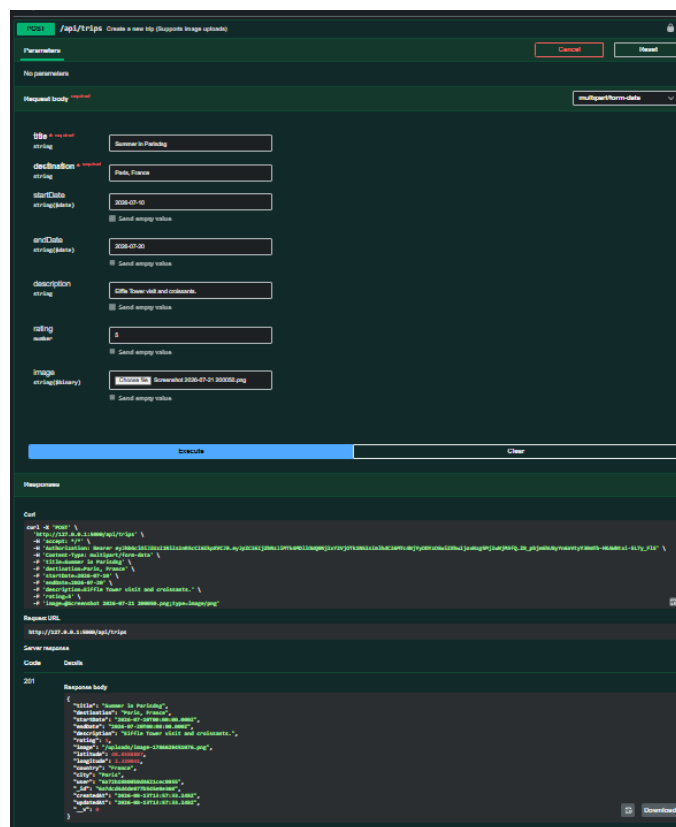


Figure 17: Swagger — POST /api/trips (Create Trip)

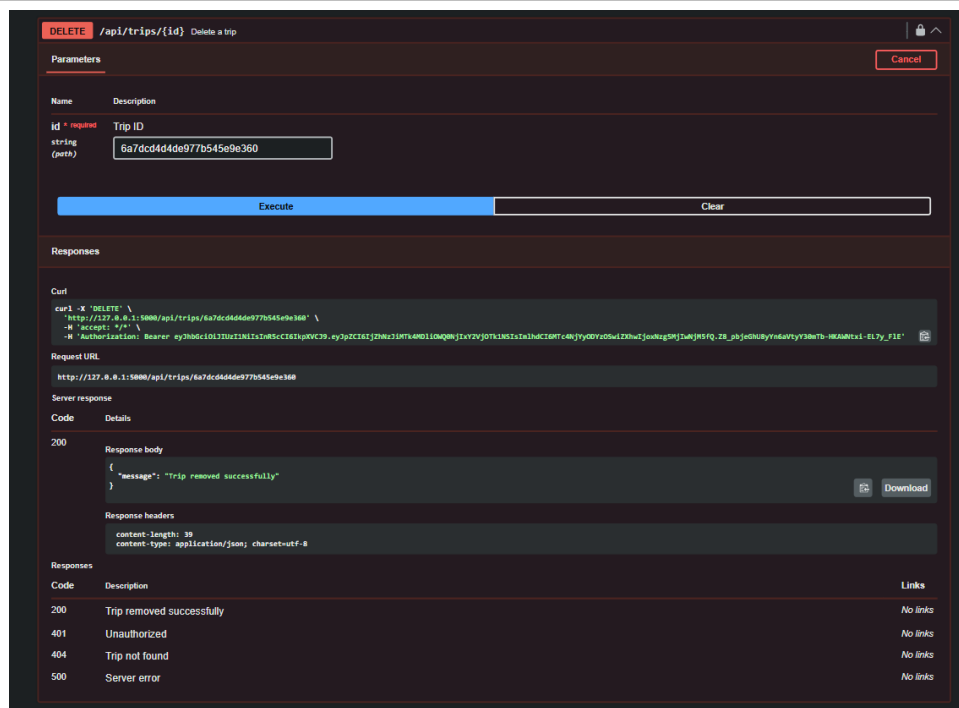


Figure 18: Swagger — DELETE /api/trips/{id} Response

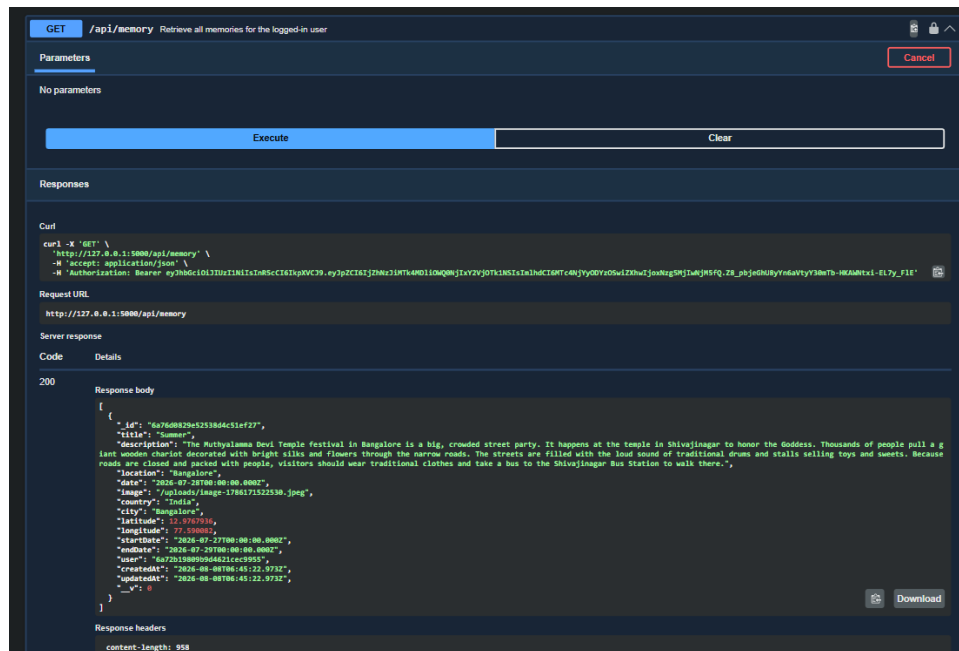


Figure 19: Swagger — GET /api/memory Response

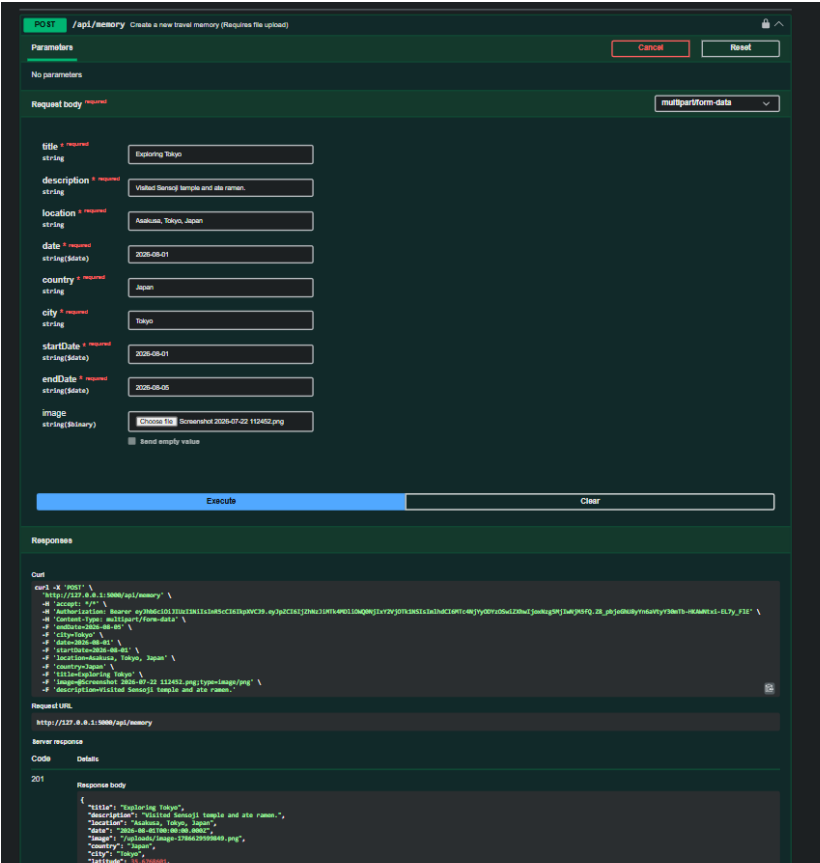


Figure 20: Swagger — POST /api/memory (Create Memory)

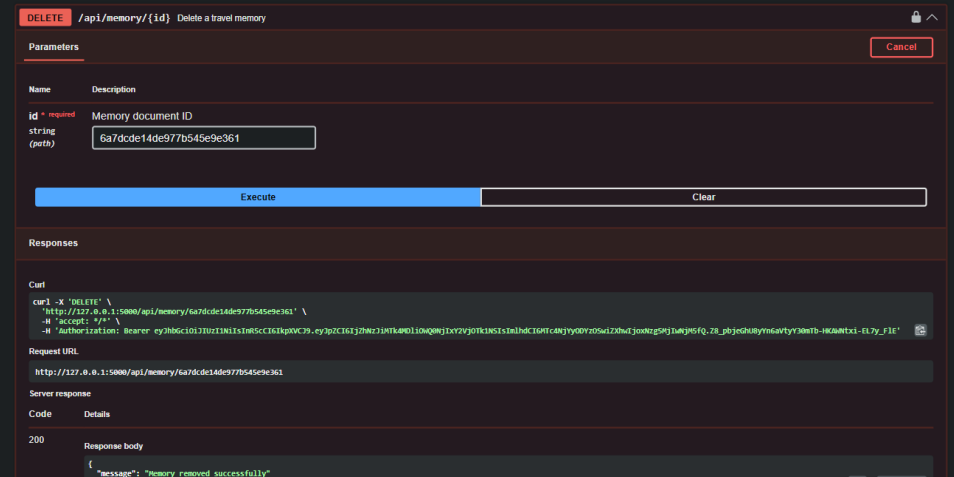


Figure 21: Swagger — DELETE /api/memory/{id} Response

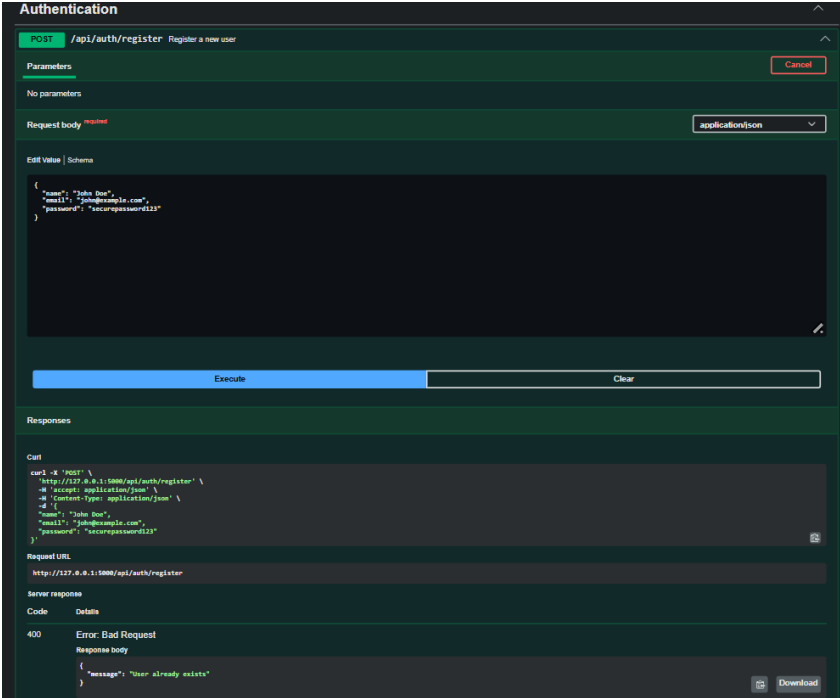


Figure 22: Swagger — POST /api/auth/register

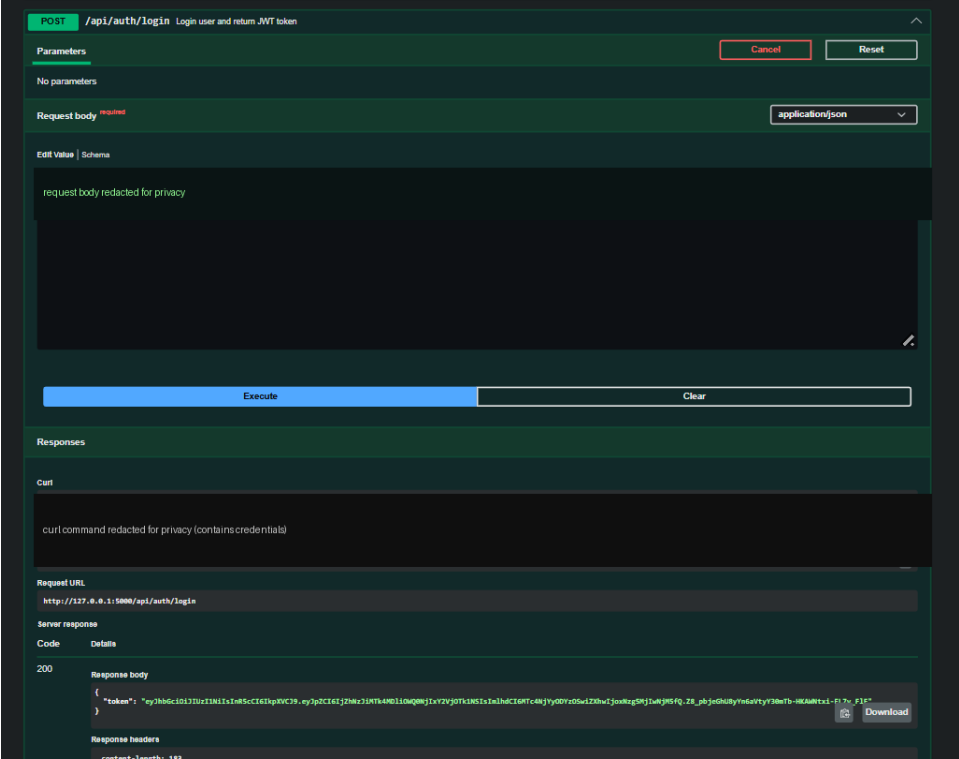


Figure 23: Swagger — POST /api/auth/login (credentials redacted)

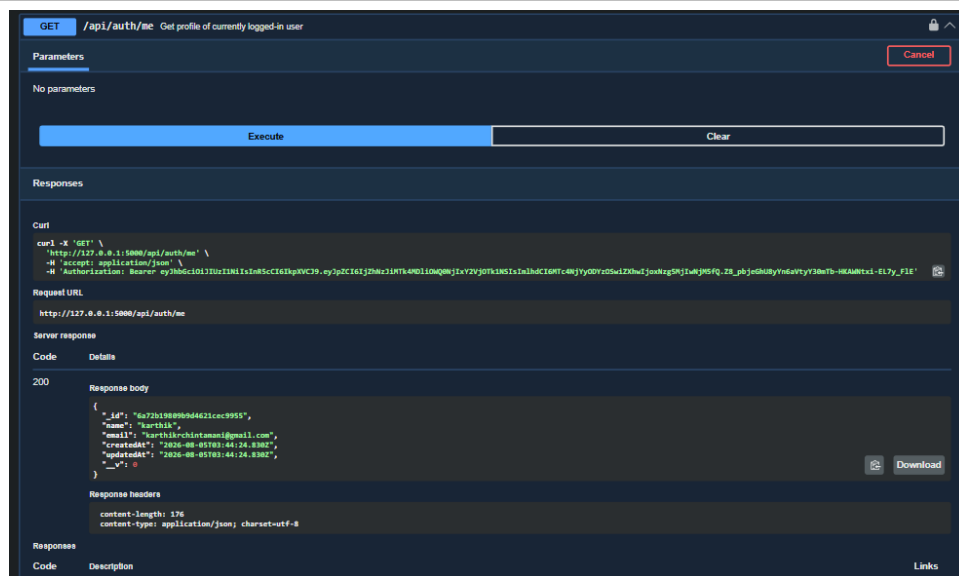


Figure 24: Swagger — GET /api/auth/me Response

Database

Trip and memory documents persisted in the local/Atlas MongoDB instance, viewed through MongoDB Compass:

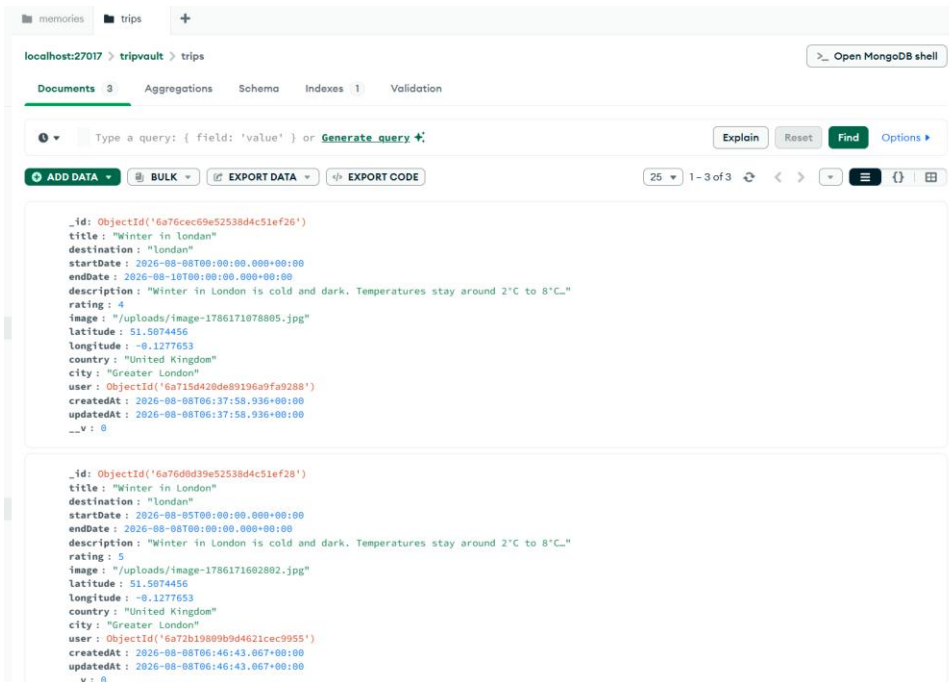


Figure 25: MongoDB — trips Collection

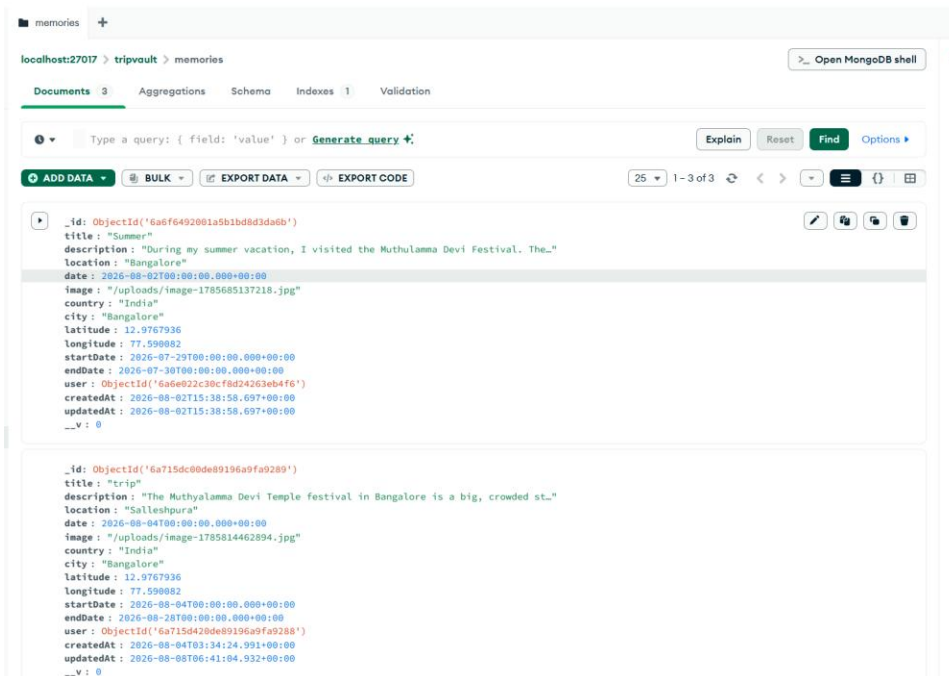


Figure 26: MongoDB — memories Collection

18. Algorithms

Create Trip Algorithm

1. Start.
2. Receive title, destination, dates, description, and rating from the client, plus an optional image file.
3. Verify the request carries a valid JWT (authentication middleware).
4. If an image file is present, let Multer store it and record its path.
5. Create a new Trip document with the supplied fields and the authenticated user's ID.
6. Save the trip to MongoDB and return it to the client.
7. End.

Update Trip Algorithm

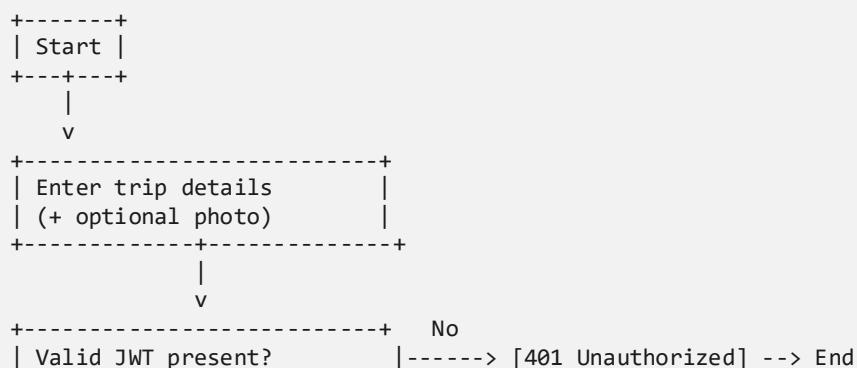
8. Start.
9. Receive the trip ID from the URL and the updated fields from the request body.
10. Look up the trip by ID, filtered by the authenticated user's ID.
11. If no matching trip is found, return a 404 error and stop.
12. Apply the supplied fields to the trip document; if a new image was uploaded, replace the stored path.
13. Save the updated trip and return it to the client.
14. End.

Delete Trip Algorithm

15. Start.
16. Receive the trip ID from the URL.
17. Attempt to find and delete a trip matching both that ID and the authenticated user's ID.
18. If no matching trip is found, return a 404 error and stop.
19. Return a success message to the client.
20. End.

19. Flowcharts

Create Trip Flowchart



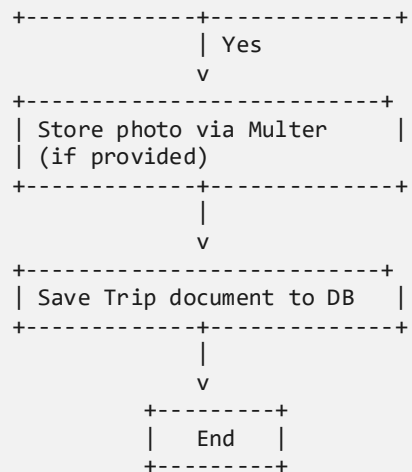


Figure 3: Create Trip Flowchart

Update Trip Flowchart

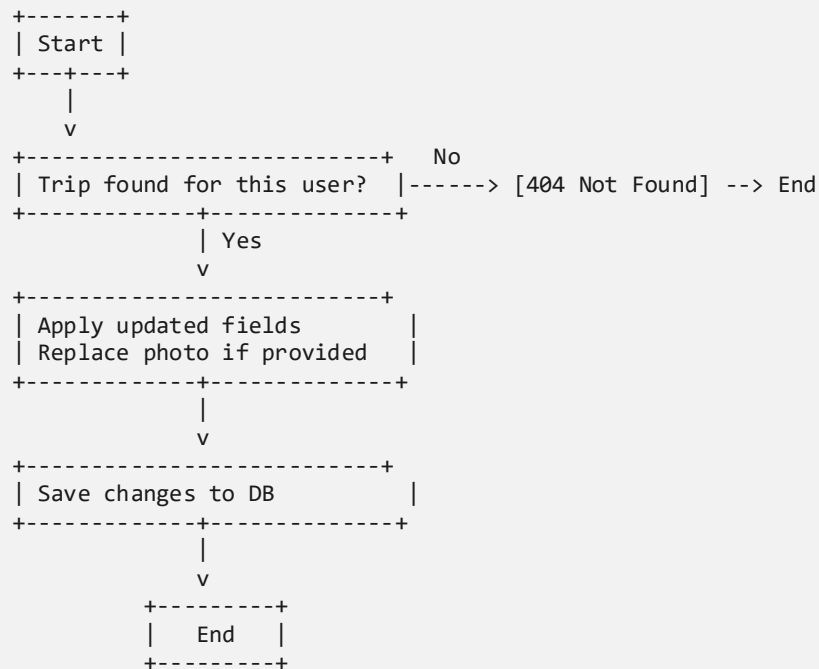
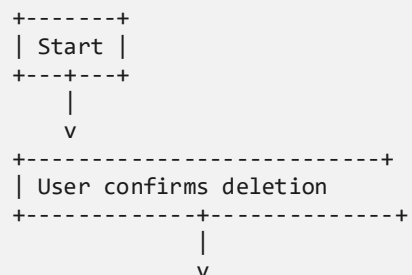


Figure 4: Update Trip Flowchart

Delete Trip Flowchart



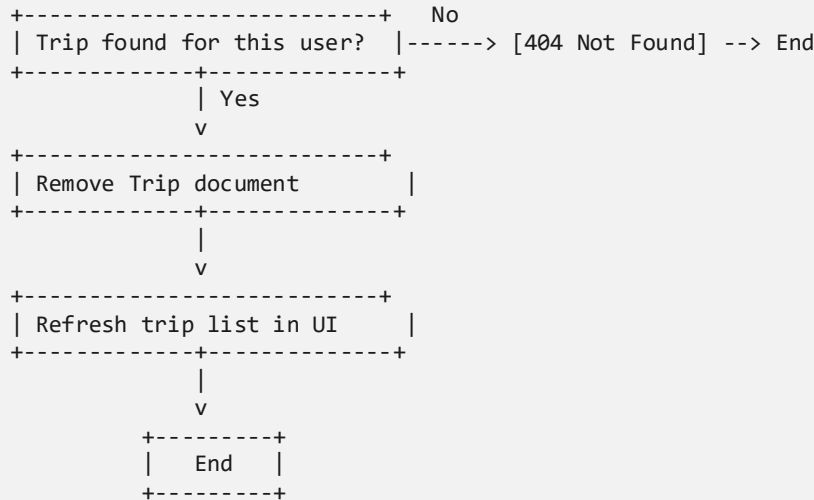


Figure 5: Delete Trip Flowchart

20. Results

By the end of Week 2, TripVault users can create, view, edit, and delete trips end-to-end. New trips — with or without a photo — are created through the Plan New Trip form and immediately appear as cards on the Trip Planner page. Every trip card correctly shows only that user's own trips, confirmed by testing with two separate accounts. Editing a trip updates its stored fields (and photo, when replaced) without affecting other trips, and deleting a trip removes it after a confirmation prompt and refreshes the list automatically. Swagger testing confirmed correct status codes for the happy path as well as for edge cases such as an unauthenticated request (401) and an attempt to access another user's trip (404). The parallel Memories module and the Dashboard summary view were also completed and verified working alongside Trip Management.

21. Advantages

- Full CRUD coverage for trips, built on the same secure, JWT-protected foundation as Week 1.
- Resource-level authorization — ownership is re-verified server-side on every update and delete, not assumed from the client.
- A single reusable trip form for both create and edit reduces frontend code duplication.
- Multer-based image handling keeps photo uploads simple while staying within the existing REST/JSON API shape.
- Automatic list refresh after every mutation keeps the UI consistent with the database at all times.
- Swagger documentation makes every endpoint self-describing and easy to test without extra tooling.

22. Future Enhancements

- Interactive World Map — highlighting visited trip destinations geographically.
- Visited Countries Tracker — automatic tallying of countries and regions explored, building on the stats already shown on the Dashboard.
- Photo Gallery — organised, multi-photo storage and display per trip, beyond the single trip image.

- Weather API Integration — historical or forecast weather data for trip destinations.
- AI Travel Recommendation — personalised destination suggestions based on trip history.
- Expense Tracker — logging and categorising trip-related expenses.
- Google Maps Integration — route mapping and destination search when creating a trip.
- Travel Analytics Dashboard — deeper statistics such as distance travelled and trip frequency over time.
- Offline Mode — local caching of trips for access without an active internet connection.

23. Conclusion

Week 2 of the TripVault Virtual Internship Program has successfully delivered a complete, secure Trip Management feature on top of the authentication foundation built in Week 1. With trip creation, listing, retrieval, editing, and deletion all implemented, protected by JWT, and scoped to their owning user — and with a working React Trip Planner UI, photo upload support, and Swagger-verified API behaviour — TripVault is well positioned to move into richer, more visual features such as maps and analytics in the milestones ahead.

24. References

React Documentation — <https://react.dev/>

Node.js Documentation — <https://nodejs.org/en/docs/>

Express.js Documentation — <https://expressjs.com/>

MongoDB Documentation — <https://www.mongodb.com/docs/>

Mongoose Documentation — <https://mongoosejs.com/docs/>

Multer Documentation — <https://github.com/expressjs/multer>

Swagger / OpenAPI Documentation — <https://swagger.io/docs/>

JWT (JSON Web Token) Documentation — <https://jwt.io/introduction>

25. Appendix

Sample JSON Request (Create Trip)

```
{
  "title": "Winter in London",
  "destination": "London",
  "startDate": "2026-08-05",
  "endDate": "2026-08-08",
  "description": "Winter in London is cold and dark. Temperatures stay around 2°C to 8°C...",
  "rating": 5
}
```

Note: image is sent as a file field (multipart/form-data), alongside the JSON fields above.

Sample JSON Response (Create Trip)

```
{
  "_id": "6a76d0d39e52538d4c51ef28",
  "title": "Winter in London",
  "destination": "london",
  "startDate": "2026-08-05T00:00:00.000Z",
  "endDate": "2026-08-08T00:00:00.000Z",
  "rating": 5,
  "image": "/uploads/image-1786171602802.jpg",
  "user": "6a72b19809b9d4621cec9955",
  "createdAt": "2026-08-08T06:46:43.067Z",
  "updatedAt": "2026-08-08T06:46:43.067Z"
}
```

Sample JSON Response (Delete Trip)

```
{
  "message": "Trip removed successfully"
}
```